Python Dictionaries: A Comprehensive Guide

A **Dictionary** is a collection which is ordered (as of Python 3.7+), changeable (mutable), and does not allow duplicates. Dictionaries are written with curly brackets `{}`, and have **keys** and **values**.

1. Creating and Accessing Dictionaries

a. Declaration

Dictionaries store data in `key: value` pairs. Keys must be unique and immutable (like strings, numbers, or tuples).

```
# Empty dictionary
user = {}

# Standard dictionary
car = {
  "brand": "Ford",
  "model": "Mustang",
  "year": 1964
}

# Using the dict() constructor
person = dict(name="John", age=36, country="Norway")
```

b. Accessing Items

You can access the items of a dictionary by referring to its key name, inside square brackets or using the `get()` method.

```
car = {"brand": "Tesla", "model": "Model S", "year": 2023}

# Method 1: Square Brackets
print(car["brand"])  # Output: Tesla

# Method 2: get() method (safer, returns None if key doesn't exist)
print(car.get("model"))  # Output: Model S

# Getting all keys or values
print(car.keys())    # Output: dict_keys(['brand', 'model', 'year'])
print(car.values())  # Output: dict_values(['Tesla', 'Model S', 2023])
```

2. Nested Dictionaries: Complex Data Structures

A nested dictionary is a dictionary that contains other dictionaries as values. This is common when representing hierarchical data.

a. Example 1: Creating a Nested Structure

```
# A dictionary representing a company department
employees = {
    "emp1": {
        "name": "John",
        "role": "Developer",
        "skills": ["Python", "AWS"]
    },
    "emp2": {
        "name": "Sarah",
        "role": "Designer",
        "skills": ["Figma", "UI/UX"]
    }
}

# Accessing Nested Data
print(employees["emp1"]["name"])     # Output: John
print(employees["emp2"]["skills"][0]) # Output: Figma
```

3. Operations with Strings and Numbers

Dictionaries often store strings and numbers that require manipulation. Here are common examples:

a. String Operations in Dictionaries

```
user_profile = {"username": "  johndoe77  ", "email": "ADMIN@example.com"}

# Cleaning string data
user_profile["username"] = user_profile["username"].strip()
user_profile["email"] = user_profile["email"].lower()

print(user_profile) # {'username': 'johndoe77', 'email': 'admin@example.com'}
```

b. Number Operations in Dictionaries

```
inventory = {"apples": 50, "price": 1.5}

# Performing arithmetic
```

```
inventory["apples"] += 10                # Incrementing stock
inventory["total_value"] = inventory["apples"] * inventory["price"]

print(inventory) # {'apples': 60, 'price': 1.5, 'total_value': 90.0}
```

4. Dictionary Methods: Implementation Examples

```
# Initialize a sample dictionary
data = {"name": "Alice", "age": 25, "city": "New York"}

# 1. get(key, default) - Safely access values
print(data.get("name"))         # Output: Alice

# 2. keys(), values(), items() - View objects
print(data.keys())              # dict_keys
print(data.values())            # dict_values
print(data.items())             # dict_items

# 3. update() - Merges or updates
data.update({"age": 26, "email": "alice@example.com"})

# 4. setdefault() - Ensures key exists
job = data.setdefault("job", "Developer")

# 5. pop() & popitem() - Removal
data.pop("age")                 # Removes specific key
data.popitem()                  # Removes last item

# 6. copy(), fromkeys(), clear()
backup = data.copy()
new_dict = dict.fromkeys(['a', 'b'], 0)
data.clear()
```

5. Multiple Choice Questions (MCQs)

Q1. Which of the following is the correct way to create a dictionary?

- B) `d = { "a": 1 }`

Q2. What happens if you try to access a non-existent key using `dict["non_existent"]`?

- C) It raises a `KeyError`.

Q3. Which method would you use to merge two dictionaries together?

- D) `update()`

(Refer to earlier sections for full MCQ list and keys)

6. Dictionary Interview Programming Questions

Basic Level Questions

#	Problem Description	Sample Input	Expected Output
1	Create a dictionary	Keys: `a, b` , Vals: `1, 2`	`{'a': 1, 'b': 2}`
2	Access key value	`d={'x': 10}` , key= `'x'`	`10`
3	Change value	`d={'y': 5}` , new val= `10`	`{'y': 10}`
4	Check if key exists	`d={'id': 1}` , key= `'id'`	`True`
5	Iterate and print keys	`{'name': 'Bob'}`	`name`

Intermediate to Advanced Level

#	Problem Description	Sample Input	Expected Output
6	Count Characters	`"hello"`	`{'h':1, 'e':1, 'l':2, 'o':1}`
7	Merge & Sum	`{'a':10}, {'a':20}`	`{'a': 30}`
8	Filter Dictionary	`{'a':1, 'b':5}, val>2`	`{'b': 5}`
9	Invert Dictionary	`{'a':1, 'b':2}`	`{1:'a', 2:'b'}`
10	Group by Length	`['apple', 'bat']`	`{5:['apple'], 3:['bat']}`